

Project Update Report: Midpoint Progress on Mystery_Solver: a Bayesian Network-Based Interactive Mystery Solver

Group 8

Abstract—This report covers the midpoint progress of **Mystery_Solver**, a course project that demonstrates probabilistic reasoning through a Bayesian Network-based mystery-solving application. The system estimates the most likely culprit in a fictional manuscript theft by combining prior suspect probabilities with user-supplied clues—forced entry, alibis, fingerprints, and security footage. So far, the core problem has been formulated, the software stack finalized, the Bayesian model built, and a working interactive prototype delivered. The backend runs on Python and **pgmpy**; the frontend uses Streamlit. The remaining work covers environment reproducibility, interface integration, CPD documentation, and testing.

Index Terms—Bayesian network, project update, probabilistic inference, Streamlit, pgmpy, educational AI application

I. INTRODUCTION

Reasoning under uncertainty is a core problem in artificial intelligence. Most real-world systems must make decisions from incomplete or ambiguous information, and Bayesian Networks are a well-established way to handle this—they encode dependencies between variables as a directed probabilistic graph and support posterior inference from observed evidence [1]. The formalism is mathematically grounded and, crucially, still readable enough to trace what the model is actually doing, which makes it a good fit for an educational project.

Mystery_Solver is an interactive application that puts this reasoning in front of a user through a fictional scenario. The user enters clues, and the model updates the probability that each suspect is guilty. The aim is a working browser-based tool that demonstrates the core ideas clearly—not a research system. This report describes where the project stands at the halfway point.

II. PROJECT OBJECTIVE AND CURRENT SCOPE

The scenario involves a stolen manuscript and three suspects: the Scholar, the Butler, and the Cat Burglar. Given clues from the user, the system computes a posterior distribution over these suspects, illustrating how prior belief and observed evidence interact in Bayesian inference [2].

The current version uses a single hidden cause variable, GuiltyParty, with observable evidence nodes for forced entry, suspect-specific alibis, fingerprints, and security footage. The scope was kept narrow on purpose—getting the inference correct was the priority before expanding the variable set. The second half of the project shifts focus to reproducibility, cleaner documentation, and making the tool presentable as an academic artifact.

III. METHODOLOGY AND PRELIMINARY MODEL DESIGN

The project uses a discrete Bayesian Network implemented in pgmpy [3], [4]. GuiltyParty is the parent of every evidence node, so the model assumes clue variables are conditionally independent given the true culprit. The joint distribution is:

$$P(G, E_1, \dots, E_m) = P(G) \prod_{i=1}^m P(E_i | G), \quad (1)$$

where G denotes the guilty party and E_i denotes each observed clue.

The prior is currently uniform across the three suspects:

$$P(G = A) = P(G = B) = P(G = C) = \frac{1}{3}. \quad (2)$$

Conditional probability distributions (CPDs) were assigned manually based on narrative logic. Forced entry is modeled as far more likely if the Cat Burglar is guilty; a missing alibi raises the posterior for the corresponding suspect. Inference uses variable elimination, available directly in pgmpy [4]. These CPD assignments will be explained more systematically in the final report.

The network structure is shown in Fig. 1.

IV. IMPLEMENTATION PROGRESS TO DATE

The software stack is chosen and partially implemented. Python is the project language [7]. pgmpy handles the probabilistic model and Streamlit provides the browser interface [5]. Graphviz has been considered for rendering the network structure visually [6].

A working prototype exists. The application collects evidence through sidebar controls, assembles an evidence dictionary, runs posterior inference, and displays the resulting probability distribution over suspects. The main AI mechanism is therefore already functional, though the artifact is still an early prototype rather than a finished system.

V. CURRENT ISSUES AND REMAINING WORK

Several tasks remain before the final submission. The virtual environment is currently machine-specific, so a clean, reproducible setup process needs to be defined. Motive-related interface controls exist in the sidebar but are not yet wired into inference. The CPD values are heuristic and need clearer justification in writing. Automated tests and documentation are also incomplete.

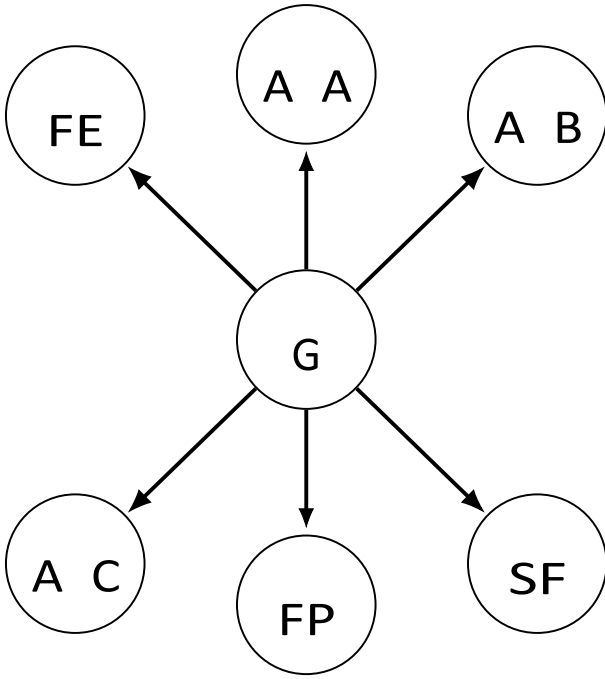


Fig. 1. Bayesian Network structure used in the current prototype. G = GuiltyParty; FE = ForcedEntry; A_A, A_B, A_C = suspect alibis; FP = Fingerprints; SF = SecurityFootage.

[7] “Python Documentation,” Python Software Foundation. [Online]. Available: <https://docs.python.org/>. Accessed: Apr. 27, 2026.

[8] “ChatGPT Overview,” OpenAI. [Online]. Available: <https://openai.com/chatgpt/overview/>. Accessed: Apr. 27, 2026.

The harder part of the project—taking an idea and turning it into a working Bayesian inference system—is done. What remains is refinement and polish.

VI. CONCLUSION

Mystery_Solver has reached a solid midpoint. The graph structure is defined, CPDs are specified, exact inference runs correctly through pgmpy, and a Streamlit interface is in place. The remaining work is well-scoped: reproducibility, interface integration, documentation, and testing. The project is on track for a complete final submission.

ACKNOWLEDGMENT

This report was prepared from direct inspection of the project code and files. ChatGPT by OpenAI was used as a writing assistance tool for organizing and refining the report text; all project-specific technical statements were verified against the implementation and official documentation [8].

REFERENCES

- [1] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.
- [2] N. L. Zhang and D. Poole, “Exploiting causal independence in Bayesian network inference,” *Journal of Artificial Intelligence Research*, vol. 5, pp. 301–328, 1996, doi: 10.1613/jair.305.
- [3] A. Ankan and A. Panda, “pgmpy: Probabilistic Graphical Models using Python,” in *Proceedings of the 14th Python in Science Conference*, 2015, pp. 6–11, doi: 10.25080/Majora-7b98e3ed-001.
- [4] “pgmpy Documentation,” pgmpy. [Online]. Available: <https://pgmpy.org/>. Accessed: Apr. 27, 2026.

- [5] “Streamlit Documentation,” Streamlit. [Online]. Available: <https://docs.streamlit.io/>. Accessed: Apr. 27, 2026.
- [6] “Graphviz Documentation,” Graphviz. [Online]. Available: <https://graphviz.org/documentation/>. Accessed: Apr. 27, 2026.